

BCA 5th Semester

PHP

Topic : Introduction to Web Development

- By Soudip Sir

1

Introduction to Web Development: Overview of web development, server vs. client-side, HTML basics, introduction to PHP and MySQL

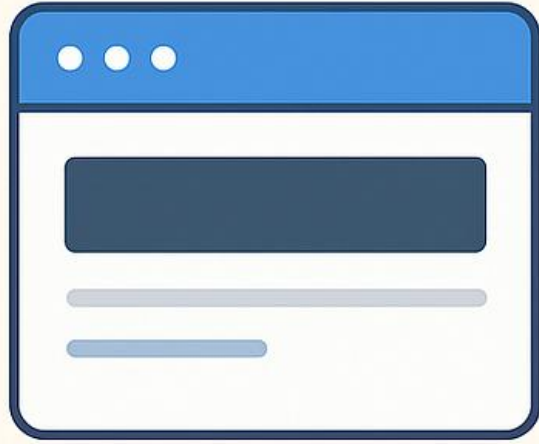
Setting up Environment: Installation of XAMPP/WAMP, introduction to Apache, MySQL, and PHP. Configuring a development environment

What is Web Development?

Web development is the process of creating and maintaining websites and web applications that run through a web browser.

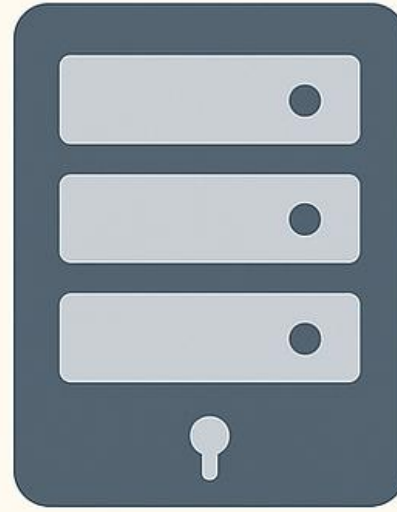
Examples:

- Google ✓
- YouTube ✓
- Facebook ✓
- Amazon ✓
- Online banking ✓
- College websites ✓
- Student portals ✓
- Online examination systems ✓



FRONTEND
(Client-Side)

✓✓



BACKEND
(Server-Side)

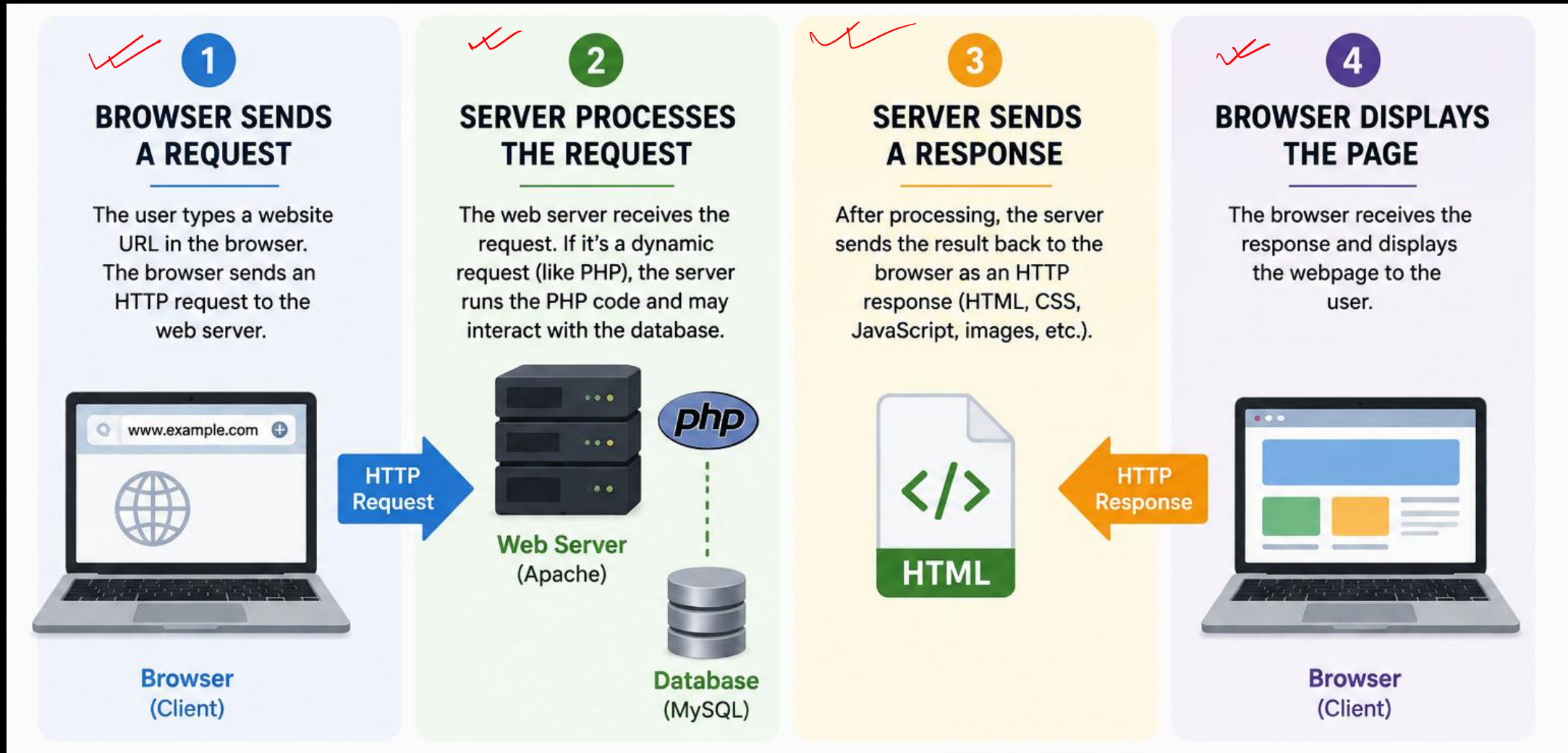
✓✓



DATABASE

✓✓

How Does a Website Actually Work?



✓ Step 1 — Browser sends a request

The browser sends a request to the web server.

Browser

|

| HTTP Request

↓

Web Server

✓ Step 2 — Server processes the request

The server receives the request and determines what needs to be returned.

If it is a PHP application, the PHP code may need to execute.

Web Server

|

↓

PHP

|

↓

Database

✓ Step 3 — Server sends a response

The server sends the result back to the browser.

Server

|

| HTTP Response

↓

Browser

✓ Step 4 — Browser displays the page

The browser receives HTML, CSS, JavaScript, images, etc., and renders the webpage.

Client-Side vs Server-Side

Feature	Client-Side	Server-Side
Meaning ✓	Code that runs on the user's device/browser	Code that runs on the web server
Execution	Executed by the browser ✓	Executed by the server ✓
Main Purpose	Handles webpage appearance and user interactions ✓	Handles business logic, data processing, authentication, etc. ✓
Common Technologies	HTML, CSS, JavaScript ✓	PHP, Python, Java, Node.js, etc. ✓
Database Access	Generally does not directly access the database	Can communicate with databases ✓
Speed	Usually fast because it runs locally ✓	Depends on server processing and network ✓
Visibility	Code is generally accessible/inspectable by the user ✓	Source code is generally hidden from the user ✓
Security	Less suitable for sensitive operations ✓	Better suited for sensitive operations ✓
Example	Form validation, animations, button interactions ✓	Login verification, database operations ✓
Runs On	User's computer/mobile ✓	Web server ✓
Example	HTML, CSS, JavaScript ✓	PHP + MySQL ✓

What is HTML?

HTML = HyperText Markup Language

HTML is a **markup language** used to create the **structure and content of web pages**.

It tells the browser:

- What is a heading ✓
- What is a paragraph ✓
- Where an image should appear ✓
- Where a link should appear ✓
- Where a form should appear ✓
- How content is structured ✓

Basic HTML Document Structure

Every beginner should understand this structure:

```
<!DOCTYPE html>

<html>

<head>
  <title>My First Website</title>
</head>

<body>

  <h1>Hello World</h1>

  <p>Welcome to HTML.</p>

</body>

</html>
```

```
<!DOCTYPE html>
```

It tells the browser that the document uses HTML5. ✓

HTML

```
<!DOCTYPE html>
```

It is a **document type declaration**, not an HTML element.

`<html>`

It is the root element of the HTML document.



Everything except the doctype declaration is generally inside `<html>`.

 HTML

```
<html>
...
</html>
```

`<title>`



Defines the title of the webpage shown in the browser tab.

 HTML

```
<title>Great Heights</title>
```

`<head>`

The `<head>` contains information about the webpage.

For example:

HTML

```
<head>
  <title>Great Heights</title>
</head>
```

It can contain:

- `<title>`
- `<meta>`
- `<link>`
- `<style>`
- `<script>`



`<body>`

The `<body>` contains the content displayed on the webpage.

Example:

HTML

```
<body>

  <h1>Welcome</h1>

  <p>Welcome to Great Heights.</p>

</body>
```

Opening and Closing Tags

Most HTML elements have:

```
<opening-tag>  
  Content  
</closing-tag>
```

```
<p>This is a paragraph.</p>
```

But some tags don't need a closing tag.

These are called **void elements**.

Examples:

```
<br>  
<hr>  
<img>  
<input>
```

Headings

HTML provides six heading levels.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```

Paragraph

The <p> tag is used for paragraphs.

```
<p>
    Great Heights provides academic and technical
    education for students.
</p>
```

Line Break

 creates a line break.

```
<p>
    Hello Students!<br>
    Welcome to PHP with MySQL.
</p>
```

Links

The `<a>` tag is used to create hyperlinks.

```
<a href="https://example.com">Visit Website</a>
```

Images

The `<img>` tag displays an image.

```

```

Important attributes:

Attribute	Purpose
<code>src</code>	Specifies image location
<code>alt</code>	Alternative text describing the image

Lists

HTML supports different types of lists.

Unordered List

Uses `<ul>`.

 HTML

```
<ul>
  <li>PHP</li>
  <li>MySQL</li>
  <li>HTML</li>
</ul>
```

Output:

- PHP
- MySQL
- HTML

Ordered List

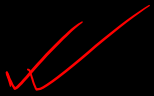
Uses `<ol>`.

 HTML

```
<ol>
  <li>HTML</li>
  <li>PHP</li>
  <li>MySQL</li>
</ol>
```

Output:

1. HTML
2. PHP
3. MySQL



HTML Comments

Comments are written using:

```
<!-- This is a comment -->
```

The browser does not display the comment as webpage content.

<!-- -->

HTML Forms

Forms are **very important for PHP** because forms allow users to send data to the server.

```
<form>

    <label>Name:</label>
    <input type="text">
                    
    <br><br>

    <label>Email:</label>
    <input type="email">
                    

    <br><br>

    <button>Submit</button>

</form>
```

Input Types

The <input> element supports different types.

```
<input type="text"> ✓
```

```
<input type="email"> ✓
```

```
<input type="password"> ✓
```

```
<input type="number"> ✓
```

```
<input type="date"> ✓
```

```
<input type="file"> ✓
```

```
<input type="checkbox"> ✓
```

```
<input type="radio"> ✓
```

HTML vs PHP

HTML	PHP
Markup language ✓	Server-side scripting language ✓
Creates webpage structure ✓	Adds dynamic/server-side functionality ✓
Runs/rendered by browser ✓	Executed on server ✓
Usually saved as .html ✓	Usually saved as .php ✓
Cannot directly perform server-side database operations ✓	Can communicate with databases ✓
Example: Hello	Example: echo "Hello";

What is PHP?

PHP is a **server-side scripting language** mainly used for web development.

PHP originally stood for **Personal Home Page**, while today PHP is commonly described as **PHP: Hypertext Preprocessor**.

It is:

- Server-side ✓
- Open-source ✓
- Widely used for web development ✓
- Embedded easily with HTML ✓
- Capable of working with databases ✓
- Used to create dynamic webpages ✓

Why Do We Need PHP?

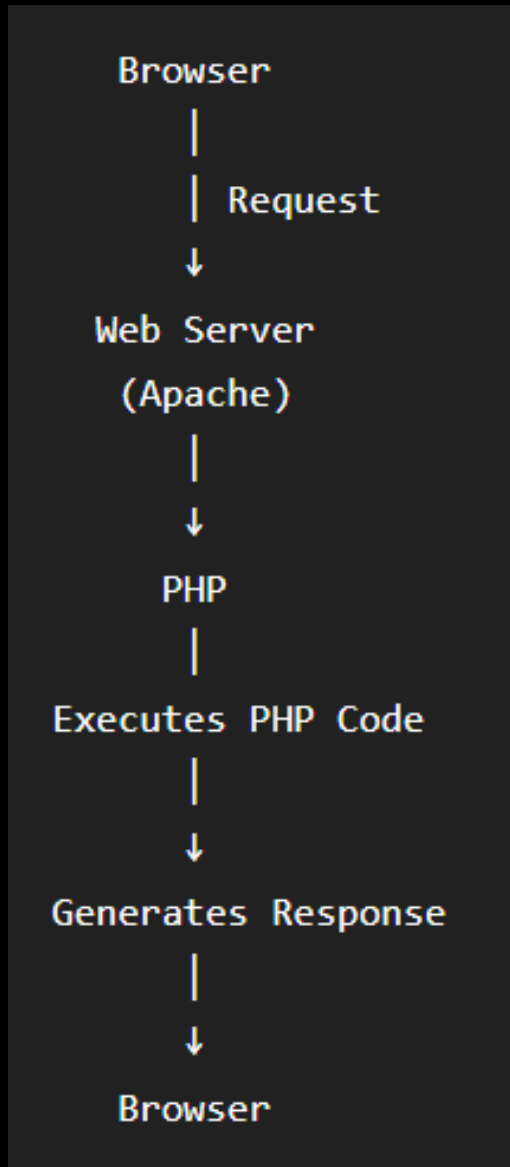
HTML can create a webpage:

But HTML alone cannot easily perform server-side operations such as:

- User login ✓
- Registration ✓
- Database operations ✓
- Password verification ✓
- Processing form data ✓
- Generating personalized pages ✓
- Managing sessions ✓
- Uploading files ✓

PHP provides this server-side functionality. ✓

How PHP Works



PHP is a Server-Side Language

What is MySQL?

MySQL is a **Relational Database Management System (RDBMS)**.

It is used to store, organize, retrieve and manage data.

Why Do We Need MySQL?

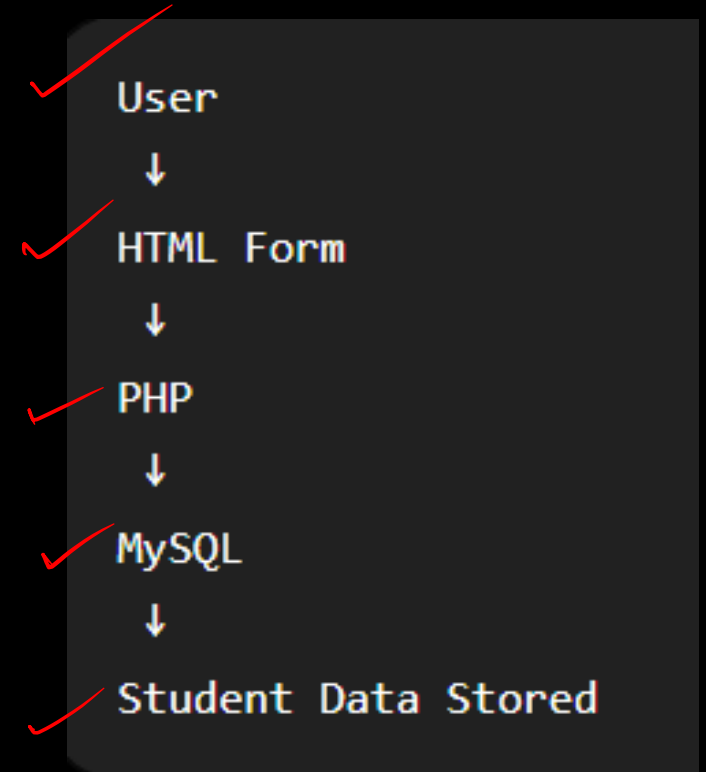
Students enter:

Name: Rahul
Email: rahul@gmail.com
Course: BCA
Semester: 5

We don't want this information to disappear after the webpage closes.

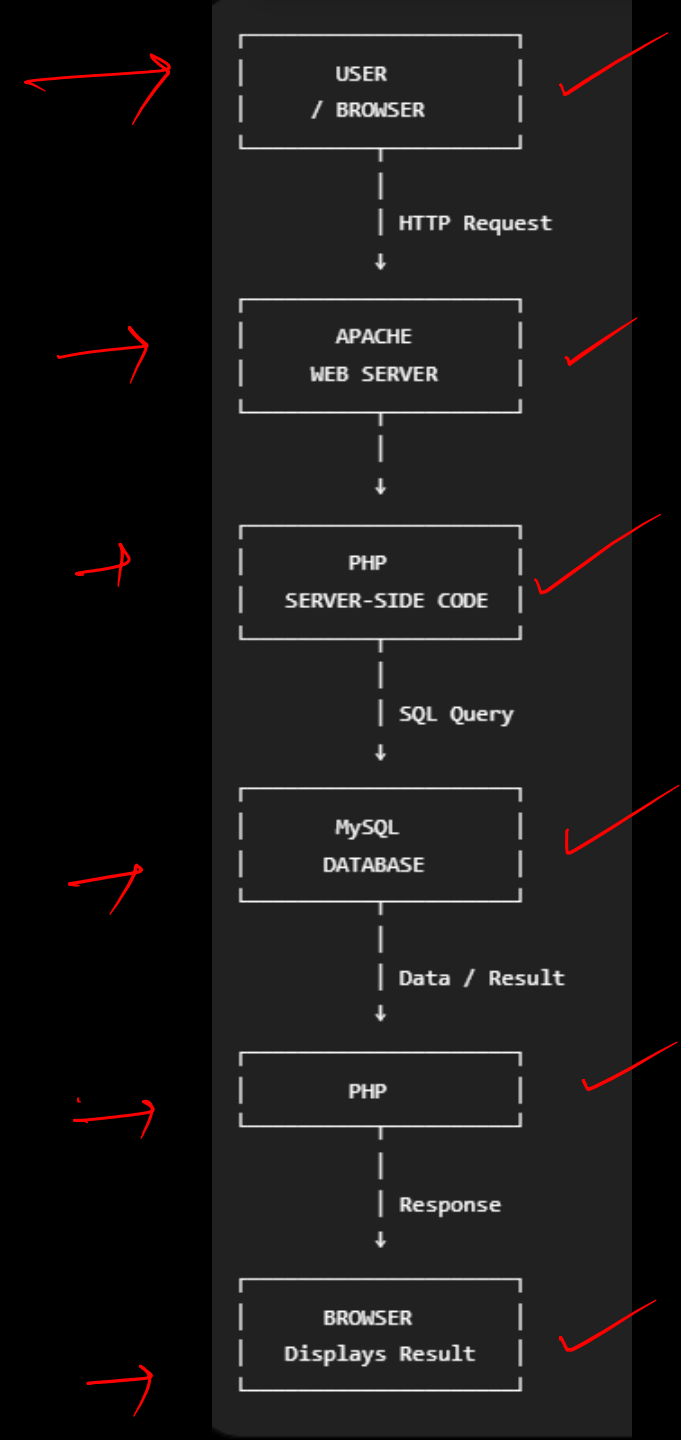
We need to **store it permanently**.

That's the job of a database.



PHP + MySQL Architecture

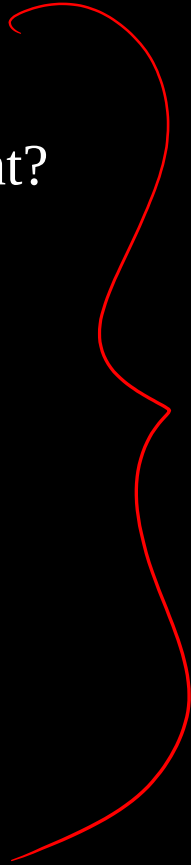
You can explain the complete concept to students using this diagram:



Setting Up PHP Development Environment

This is the **second major part of Module 1**. Now that students understand PHP, MySQL, client/server and how a website works, we need to create an environment where they can **actually run PHP programs on their computer**.

The main topics are:

- 1.What is a development environment?
 - 2.XAMPP
 - 3.WAMP
 - 4.Apache
 - 5.PHP
 - 6.MySQL/MariaDB
 - 7.phpMyAdmin
 - 8.Installing XAMPP
 - 9.Starting Apache and MySQL
 - 10.Running the first PHP program
- 

1. What is a Development Environment?

A **development environment** is a collection of software and tools that allows a developer to create, test and run applications.

For PHP development, we need:



PHP
+
Web Server
+
Database

Instead of installing everything separately, we can use packages such as:

XAMPP or **WAMP**

2. Why Do We Need XAMPP/WAMP?

Remember:

PHP is a server-side language.

Therefore, we cannot normally just double-click a `.php` file and expect it to execute.

We need:

Browser ✓



Web Server ✓



PHP ✓



Output ✓

For database applications, we additionally need:

PHP



MySQL

XAMPP/WAMP provides these components in one local development environment.

3. What is XAMPP?

XAMPP is a free and open-source local development environment commonly used for PHP development.

The name is traditionally explained as:

Letter	Meaning
X	Cross-platform ✓
A	Apache ✓
M	MySQL ✓
P	PHP ✓
P	Perl ✓

Note: Modern XAMPP distributions may bundle **MariaDB** instead of MySQL, but for your syllabus and PHP learning, the database concepts and SQL usage are closely related.

4. What is WAMP?

WAMP is another local development environment for Windows.

It commonly refers to:

W → Windows
A → Apache
M → MySQL/MariaDB
P → PHP

So the basic idea is:

XAMPP → Cross-platform package
WAMP → Windows-focused package

Both can be used for PHP development.

5. XAMPP vs WAMP

Feature	XAMPP	WAMP
Full Form	X = Cross-platform, A = Apache, M = MySQL/MariaDB, P = PHP, P = Perl	Windows, Apache, MySQL/MariaDB, PHP
Operating Systems	Windows, Linux, macOS	Mainly Windows
Web Server	Apache	Apache
PHP	Yes	Yes
Database	MySQL/MariaDB depending on distribution	MySQL/MariaDB depending on distribution
Ease of Setup	Easy	Easy
Suitable for Beginners	Yes	Yes
Used in This Course	Recommended	Alternative

6. Components We Need

When we install XAMPP, the important components for our PHP course are:

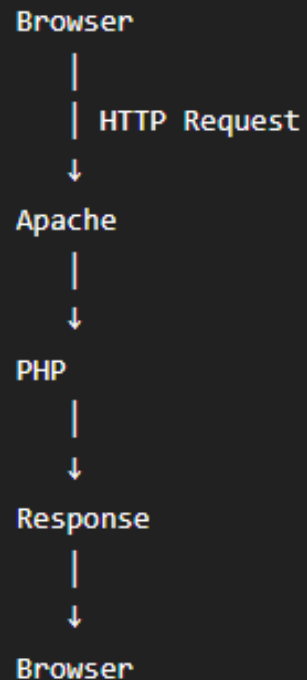
XAMPP
Apache
PHP
MySQL / MariaDB
phpMyAdmin

7. What is Apache?

Apache HTTP Server is a web server.

Its primary job is to receive HTTP requests from browsers and serve web resources.

For example:



Example

You enter:

`http://localhost/index.php`

Apache receives the request and helps route the PHP file for processing.

8. What is PHP?

PHP is the **server-side scripting language**.

Suppose we have:

```
</> PHP

<?php

echo "Hello Students";

?>
```

Apache handles the web request, while the PHP interpreter executes the PHP code.

The browser eventually receives the generated output:

```
Hello Students
```

So:

```
Apache → Handles Web Request
PHP    → Executes PHP Code
```

✓ 9. What is MySQL?

MySQL is a **Relational Database Management System (RDBMS)**.

It stores application data.

For example:

Students

ID	Name	Course
----	------	--------

1	Rahul	BCA
---	-------	-----

2	Priya	BCA
---	-------	-----

3	Amit	BCA
---	------	-----

PHP can communicate with the database to:

- Insert data
- Retrieve data
- Update data
- Delete data

These operations are collectively known as **CRUD**.

10. What is phpMyAdmin?

phpMyAdmin is a web-based graphical interface used to manage MySQL/MariaDB databases.

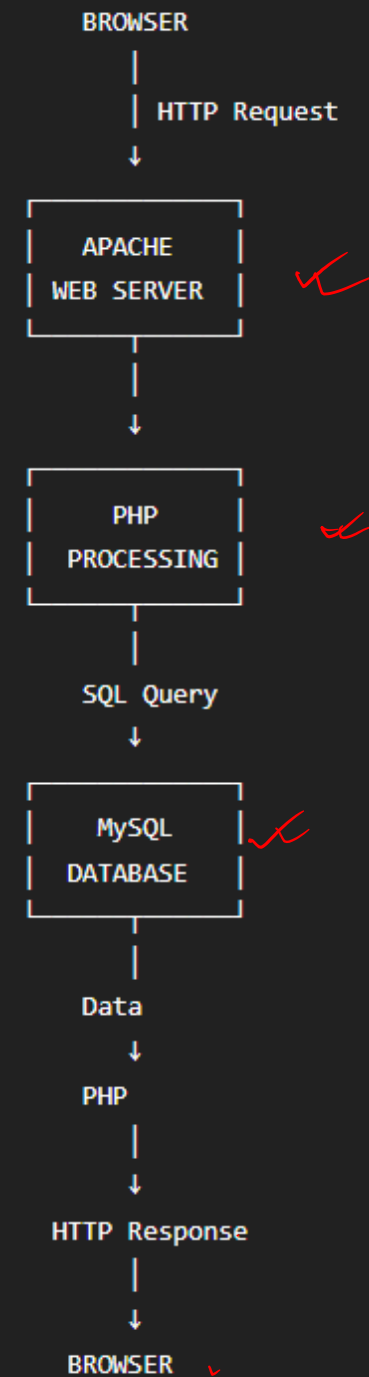
Instead of performing every database operation from a command line, we can use phpMyAdmin to:

- Create databases
- Create tables
- Insert records
- View records
- Edit records
- Delete records
- Execute SQL queries

Usually, after starting Apache, you can access it through:

```
http://localhost/phpmyadmin/
```

11. How All Components Work Together



12. Installing XAMPP on Windows

For practical classes, follow these steps.

✓ Step 1 — Download XAMPP

Download the appropriate XAMPP installer for your operating system from the official Apache Friends website.

✓ Step 2 — Run the Installer

Open the downloaded installer.

Windows may show a security confirmation.

Choose **Yes** to continue.

✓ 13. Select Components

During installation, you'll see a component selection screen.

For this course, the important components are:

- ☒ Apache
- ☒ MySQL / MariaDB
- ☒ PHP
- ☒ phpMyAdmin

Other components are not necessary for basic PHP + MySQL development.

14. Choose Installation Location

A common installation location is:

```
C:\xampp
```

Then your important directories will look like:

```
C:\xampp
|
├─ apache
├─ mysql
├─ php
├─ htdocs
└─ phpMyAdmin
```



The most important folder for our PHP projects is:

```
C:\xampp\htdocs
```

15. What is `htdocs`?

`htdocs` is the directory where Apache serves web files in a typical XAMPP setup.

For example:

```
C:\xampp\htdocs
```

Create a project:

```
C:\xampp\htdocs\php-course
```

Then:

```
php-course
|
├─ index.php
├─ about.php
└─ contact.php
```

You can access it through:

```
http://localhost/php-course/
```


16. Start XAMPP Control Panel

After installation, open:

XAMPP Control Panel

You'll see something similar to:

Module	Actions
Apache	[Start]
MySQL	[Start]

For our course, start:

Apache
MySQL

17. Starting Apache

Click:

Apache → Start

If Apache starts successfully, it will show as running in the XAMPP Control Panel.

Apache is now acting as your **local web server**.

18. Starting MySQL

Click:

MySQL → Start

Now the database server is running locally.

So our environment is:

Apache → Running
PHP → Available
MySQL → Running

19. Testing Apache

Open your browser.

Enter:

`http://localhost/`

If Apache is working correctly, you should get the XAMPP/local server page.

This confirms:

Apache is working.

20. Testing phpMyAdmin

Now open:

`http://localhost/phpmyadmin/`



If phpMyAdmin opens successfully, you can manage your MySQL/MariaDB databases through the browser.

This confirms:

Your database environment is accessible.

21. Create Your First PHP Project

Go to:

C:\xampp\htdocs

Create a folder:

php-course

Inside it, create:

index.php

So the structure is:

C:\xampp\htdocs\php-course\index.php

22. Write Your First PHP Program

Open `index.php` in VS Code.

Write:

```
<? PHP
<?php
echo "Hello Students!";
?>
```

Save the file.

23. Run Your PHP Program

Make sure:

Apache → Running

Then open:

<http://localhost/php-course/>

The browser should display:

Hello Students!

🎉 You have successfully executed your first PHP program.

Configuring a development environment in theory based

Configuring a development environment means installing and properly setting up the web server, PHP interpreter, database server, database management tools, and code editor so that PHP applications can be developed and tested locally.

Configuring a PHP Development Environment

After installing **XAMPP**, the next step is to configure the environment so that we can **create, run, test, and manage PHP + MySQL applications locally**.

1. What is a Development Environment?

A **development environment** is a set of software tools and configurations required to **develop, test, debug, and run an application**.

For PHP development, the environment must provide:

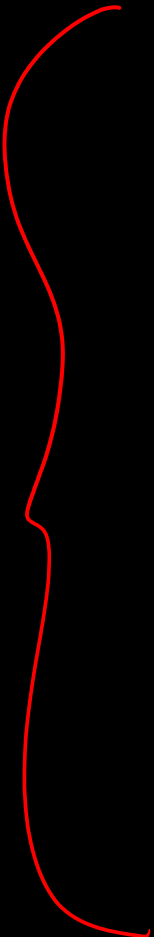
- A **web server**
- A **PHP interpreter**
- A **database server**
- A **database management tool**
- A **code editor**

The basic setup is:

```
graph TD; A[Code Editor] --> B[PHP Application]; B --> C[Web Server]; C --> D[PHP Interpreter]; D --> E[Database Server];
```

Code Editor
↓
PHP Application
↓
Web Server
↓
PHP Interpreter
↓
Database Server

2. Components of a PHP Development Environment



Component	Purpose
Code Editor	Used to write and edit PHP code ✓
Apache	Acts as the web server ✓
PHP	Executes server-side PHP code ✓
MySQL/MariaDB	Stores and manages application data ✓
phpMyAdmin	Provides a graphical interface to manage databases ✓
Browser	Used to access and test the web application ✓
XAMPP/WAMP	Provides a ready-made local development environment ✓

3. Configuration Process

Configuring a PHP development environment generally involves the following steps:

Step 1 — Install Required Software ✓

Install a package such as:

- XAMPP
- WAMP

These provide the major components required for PHP development.

✓ Step 2 — Configure the Web Server

The web server, commonly **Apache**, must be installed and configured to:

- Receive HTTP requests
 - Serve website files
 - Process PHP requests through PHP
 - Provide access to local projects
-

Step 3 — Configure PHP

PHP must be available to the web server so that `.php` files can be executed.

PHP configuration can be controlled through its configuration file:

```
php.ini
```

This file contains settings related to:

- Error reporting
- Memory usage
- File uploads
- Execution time
- Extensions
- Date and time settings

Step 4 — Configure the Database

A database server such as **MySQL/MariaDB** is configured to store application data.

For example:

Database



Tables



Records

PHP can communicate with the database using database extensions such as **MySQLi** or **PDO**.

Step 5 — Configure the Web Root

A web root is the directory from which the web server serves website files.

In a typical XAMPP setup:

```
htdocs
```

PHP projects are placed inside this directory.

Example:

```
htdocs
├──
└── myproject
    ├──
    └── index.php
```

Step 6 — Configure Database Management

phpMyAdmin can be used to manage MySQL/MariaDB databases through a web interface.

It allows developers to:

- Create databases
- Create tables
- Insert records
- Update records
- Delete records
- Execute SQL queries

Step 7 — Test the Environment

After configuration, each component should be tested.

```
Apache → Web server working
PHP → PHP code executing
MySQL → Database working
phpMyAdmin → Database management working
Browser → Application accessible
```

4. Localhost

A PHP development environment is often configured to run locally.

localhost refers to the computer on which the application is currently running.

For example:

```
http://localhost/
```

A project may be accessed as:

```
http://localhost/myproject/
```

The general flow is:

```
graph TD; Browser --> localhost; localhost --> Apache; Apache --> PHP; PHP --> Response; Response --> Browser;
```

5. Port Configuration

A **port** is a logical communication endpoint used by network services.

Common default ports are:

Service	Typical Port	
HTTP / Apache	80	✓
HTTPS / Apache	443	✓
MySQL/MariaDB	3306	✓

6. Configuration of PHP Extensions

PHP functionality can be extended through **PHP extensions**.

For PHP + MySQL development, database-related extensions are important.

Examples:

```
mysqli  
PDO
```

Extensions can be enabled or configured through the PHP configuration.

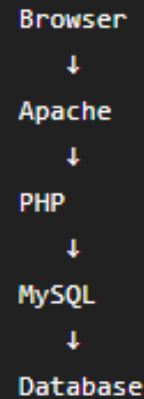
Conceptually:

```
PHP  
|  
├─ Core functionality  
├─ MySQLi  
├─ PDO  
├─ OpenSSL  
└─ Other extensions
```

7. Why Configuration is Important

Proper configuration ensures that all components can communicate correctly.

For example:



If one component is incorrectly configured, the application may not work.

Examples:

- Apache not running → PHP page cannot be served through the local web server.
- PHP not configured → PHP code cannot be executed.
- MySQL not running → Database operations fail.
- Incorrect database configuration → PHP cannot connect to the database.
- Port conflict → A server may fail to start.

8. Overall PHP Development Environment

